SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems
COURSE CODE: CSA0403

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements **Assessment Tool Weightage:** 30%

QUESTIONS: 10 **Total Marks: 50**

CASE STUDY:

1 A multinational software company develops a large ERP application consisting of 120 dynamically linked libraries. During execution, users complain that startup time is high while memory utilization remains low. As the system architect, analyze whether static linking or dynamic linking is the better choice. Recommend an optimized loading strategy by considering compile time, load time, execution time, and shared libraries. Justify your decision with suitable memory diagrams.

2 A cloud server simultaneously executes AI applications, database services, and web applications. Although 128 GB RAM is installed, memory allocation failures occur frequently. Investigate the causes by analyzing dynamic storage management, external fragmentation, and internal fragmentation. Design a memory management strategy that minimizes memory wastage while maintaining high throughput.

3 An airline reservation system experiences severe delays whenever thousands of customers book tickets simultaneously. The operating system frequently swaps processes between RAM and disk. Analyze how swapping affects CPU utilization and response time. Propose modifications to reduce swapping overhead without upgrading hardware.

4 A hospital management system allocates memory using contiguous memory allocation. After months of continuous operation, several large applications fail to load despite sufficient free

memory. Investigate the reason using fragmentation concepts. Compare First Fit, Best Fit, Worst Fit, and Next Fit strategies, and recommend the most appropriate allocation technique for this scenario.

5 A banking application initially uses contiguous memory allocation but is later migrated to paging. Compare the performance of both approaches under heavy transaction loads. Evaluate page table overhead, fragmentation, memory utilization, and execution efficiency before recommending the migration strategy.

6 A gaming company develops an open-world game with multiple independent modules such as graphics, AI, physics engine, and audio processing. Analyze how segmentation can improve memory organization compared to paging. Design an appropriate segment table and explain how logical addresses are translated into physical addresses.

7 A software development company uses both segmentation and paging in its operating system. During debugging, developers notice increased address translation time. Analyze the advantages and disadvantages of segmentation with paging compared to pure paging and pure segmentation. Recommend an optimized architecture for the organization.

8 A mobile operating system allows users to run multiple applications simultaneously, exceeding the available physical memory. Analyze how virtual memory enables efficient multitasking. Explain the consequences if virtual memory is disabled, and propose improvements for better memory performance.

9 An online video editing application loads only required portions of videos into memory. Users report occasional delays when editing large files. Analyze how demand paging influences system performance. Suggest methods to reduce page faults without increasing RAM capacity.

10 A data analytics server experiences frequent page faults while processing large datasets. After monitoring memory references, the administrator notices repetitive loading and replacement of the same pages. Analyze the effectiveness of FIFO, LRU, Optimal, and Clock page replacement algorithms. Recommend the most suitable algorithm for this workload with proper justification.

11 A university examination server crashes during online examinations due to excessive page faults. Investigate whether thrashing is responsible for the failure. Analyze CPU utilization, page fault frequency, and working set size. Design an improved memory allocation policy to eliminate thrashing.

12 A cloud provider hosts hundreds of virtual machines on a single physical server. During peak hours, several VMs become unresponsive. Analyze the role of working sets in preventing thrashing. Propose a memory allocation framework that dynamically adjusts working set sizes based on workload patterns.

13 A multimedia company stores high-definition video files on HDDs while frequently accessed metadata is stored on SSDs. Analyze how disk devices influence virtual memory performance, swapping speed, and page replacement efficiency. Recommend a storage hierarchy that minimizes access latency.

14 A software company plans to redesign its operating system memory manager. The proposed system integrates dynamic linking, demand paging, segmentation, virtual memory, and SSD-based swapping. Evaluate whether this integrated approach improves overall system performance. Identify possible bottlenecks and propose architectural improvements.

15 An e-commerce platform experiences inconsistent response times during festival sales. System logs reveal excessive dynamic library loading, frequent page replacements, and swapping activity. Analyze the interaction among dynamic linking, demand paging, and swapping. Develop a comprehensive optimization strategy that improves performance while minimizing memory consumption.

16 A research laboratory develops a scientific simulation requiring large matrices exceeding available RAM. Compare the effectiveness of virtual memory, segmentation, paging, and segmentation with paging. Recommend the most efficient memory management model by considering scalability, performance, and memory protection.

17 A cybersecurity organization requires strict memory protection between user processes and kernel processes. Analyze how segmentation, paging, and segmentation with paging provide isolation and protection. Design a secure memory architecture capable of preventing unauthorized memory access while maintaining execution efficiency.

18 A fintech company is designing a next-generation operating system for financial servers. The system must support secure memory allocation, efficient page replacement, fast loading of shared libraries, and minimal disk I/O. Develop a complete memory management framework integrating dynamic storage management, paging, virtual memory, working sets, and SSD storage. Justify each design decision.

An autonomous vehicle runs perception, navigation, and object detection modules concurrently on limited embedded memory. Analyze how demand paging, page

19 replacement policies, virtual memory, and working set models influence real-time performance. Recommend a customized memory management strategy that guarantees deterministic execution with minimal latency.

A technology company plans to build an operating system for future AI-powered supercomputers. The system must efficiently support millions of concurrent processes, shared libraries, heterogeneous memory devices, SSDs, and distributed storage. Design a complete memory management architecture integrating linkers,

20 dynamic linking, dynamic storage management, swapping, contiguous allocation, paging, segmentation, segmentation with paging, virtual memory, demand paging, page replacement, thrashing prevention, working sets, and advanced disk management. Evaluate trade-offs among performance, scalability, reliability, and cost.

Question 1: Dynamic Linking vs Static Linking

Case Analysis

The ERP application contains **120 dynamically linked libraries**. Users experience **high startup time**, while memory utilization remains low. The task is to determine the better linking method and recommend an efficient loading strategy.

Analysis

- **Static Linking** combines all libraries with the executable during compilation. It provides faster execution but increases executable size and memory usage.
- **Dynamic Linking** loads libraries only when required. It reduces executable size, allows library sharing, and improves memory utilization, though startup time may be slightly higher.

Recommended Solution

Use **Dynamic Linking with Lazy Loading (Demand Loading)**. Load only essential libraries at startup and load remaining libraries when needed. Store frequently used shared libraries in memory to reduce loading time.

Justification

This approach minimizes memory usage, supports shared libraries, simplifies software updates, and improves overall application performance without increasing memory consumption.

Conclusion

For a large ERP application, **Dynamic Linking with Lazy Loading** is the most suitable solution because it provides efficient memory utilization, better scalability, and acceptable startup performance.

Question 2: Memory Allocation Failures in Cloud Server

Case Analysis

The cloud server runs **AI applications, database services, and web applications** simultaneously. Despite having **128 GB RAM**, memory allocation failures occur due to inefficient memory management and fragmentation.

Analysis

- **Dynamic Storage Management** allocates memory based on process requirements.
- **External Fragmentation** creates scattered free memory blocks, preventing large memory allocation.
- **Internal Fragmentation** wastes memory because allocated blocks are larger than required.
- These issues reduce memory utilization and system performance.

Recommended Solution

Implement **Paging** with **Dynamic Memory Allocation** to eliminate external fragmentation. Use **Virtual Memory** and an efficient page replacement algorithm (LRU or Clock) to improve memory utilization and system throughput.

Justification

Paging minimizes memory wastage, improves CPU utilization, supports multiple applications efficiently, and maintains high system performance.

Conclusion

Using **paging, virtual memory, and efficient memory allocation techniques** provides better memory utilization and reduces allocation failures in the cloud server.

Question 3: Swapping in Airline Reservation System

Case Analysis

The airline reservation system experiences delays when thousands of users book tickets simultaneously. The operating system frequently swaps processes between RAM and disk, reducing system performance.

Analysis

- Swapping moves processes between RAM and secondary storage when memory is insufficient.
- Frequent swapping increases disk I/O, reduces CPU utilization, and increases response time.
- This leads to slower transaction processing and poor user experience.

Recommended Solution

Reduce swapping by using virtual memory, efficient process scheduling, and demand paging. Optimize memory allocation and keep frequently used processes in RAM to minimize unnecessary disk access.

Justification

Reducing swapping improves CPU efficiency, decreases response time, increases throughput, and ensures smooth handling of multiple booking requests.

Conclusion

Implementing virtual memory, demand paging, and efficient memory management minimizes swapping overhead and improves the performance of the airline reservation system without upgrading hardware.

Question 4: Fragmentation in Hospital Management System

Case Analysis

The hospital management system uses **contiguous memory allocation**. After continuous operation, large applications fail to load even though sufficient free memory is available. The issue is caused by **memory fragmentation**. The task

is to compare **First Fit, Best Fit, Worst Fit, and Next Fit** allocation strategies and recommend the best one.

Analysis

- **External Fragmentation** splits free memory into small blocks, preventing large applications from loading.
- **First Fit** allocates the first suitable memory block and is fast.
- **Best Fit** allocates the smallest suitable block but creates many small fragments.
- **Worst Fit** allocates the largest block but wastes memory.
- **Next Fit** searches from the last allocated position, reducing search time.

Recommended Solution

Use **First Fit** with periodic **memory compaction** to reduce fragmentation and improve memory allocation efficiency.

Justification

First Fit provides faster allocation, better memory utilization, and lower overhead compared to the other techniques, making it suitable for long-running systems.

Conclusion

Using **First Fit** along with **memory compaction** minimizes fragmentation, improves memory utilization, and allows large applications to load successfully.

Question 5: Migration from Contiguous Memory Allocation to Paging

Case Analysis

A banking application initially uses **contiguous memory allocation** but later migrates to **paging**. The task is to compare both approaches under heavy transaction loads by evaluating **page table overhead, fragmentation, memory utilization, and execution efficiency**, and recommend the better method.

Analysis

- **Contiguous Memory Allocation** is simple but suffers from **external fragmentation** and poor memory utilization.
- **Paging** divides memory into fixed-size pages, eliminating external fragmentation and improving memory management. However, it introduces **page table overhead**.

Recommended Solution

Implement **Paging** with an efficient **page replacement algorithm (LRU or Clock)** to improve memory utilization and support heavy transaction processing.

Justification

Paging provides better memory utilization, reduces fragmentation, supports multitasking, and improves execution efficiency, making it suitable for banking applications.

Conclusion

Paging is the preferred memory management technique because it offers higher performance, efficient memory usage, and reliable execution under heavy transaction loads.

Question 6: Segmentation in Open-World Game

Case Analysis

An open-world game contains independent modules such as **graphics, AI, physics engine, and audio processing**. The task is to analyze how **segmentation** improves memory organization compared to paging and explain logical-to-physical address translation using a segment table.

Analysis

- **Segmentation** divides memory into logical segments such as graphics, AI, physics, and audio.

- Each segment has its own **base address** and **limit**, making memory management more organized and easier to maintain.
- Unlike paging, segmentation matches the program's logical structure, improving flexibility and protection.

Recommended Solution

Use **Segmentation** by assigning a separate segment to each game module. A **segment table** stores the base address and limit of every segment, allowing logical addresses to be translated into physical addresses efficiently.

Justification

Segmentation improves memory organization, simplifies module management, provides better protection, and allows independent loading and execution of different game components.

Conclusion

Segmentation is the most suitable memory management technique for modular game development because it offers better organization, flexibility, and efficient memory access.

Question 7: Segmentation with Paging

Case Analysis

A software development company uses **segmentation with paging** in its operating system. During debugging, developers observe increased **address translation time**. The task is to compare this approach with **pure paging** and **pure segmentation** and recommend the best architecture.

Analysis

- **Pure Paging** eliminates external fragmentation but requires page tables.
- **Pure Segmentation** provides logical memory organization but may suffer from external fragmentation.

- **Segmentation with Paging** combines the advantages of both methods, offering better memory protection and efficient allocation, though address translation takes slightly longer.

Recommended Solution

Use **Segmentation with Paging** along with an efficient **Translation Lookaside Buffer (TLB)** to reduce address translation time and improve overall performance.

Justification

This approach provides better memory protection, efficient memory utilization, reduced fragmentation, and improved flexibility for complex software development.

Conclusion

Segmentation with Paging is the most suitable architecture because it balances memory efficiency, protection, and performance, making it ideal for modern operating systems.

Question 8: Virtual Memory in Mobile Operating System

Case Analysis

A mobile operating system allows users to run multiple applications simultaneously, even when the total memory requirement exceeds the available physical memory. The task is to explain how **virtual memory** supports multitasking, the effects of disabling it, and suggest improvements for better memory performance.

Analysis

- **Virtual Memory** allows applications to use secondary storage as an extension of RAM.

- It enables multiple applications to run efficiently, even when physical memory is limited.
- If virtual memory is disabled, the system may run out of memory, reducing multitasking capability and causing application crashes.

Recommended Solution

Implement **Virtual Memory** with **Demand Paging** and an efficient **page replacement algorithm (LRU or Clock)**. Optimize memory allocation to reduce unnecessary page faults and improve performance.

Justification

Virtual memory improves multitasking, increases memory utilization, reduces application failures, and provides a smoother user experience.

Conclusion

Using **Virtual Memory** with demand paging is the best solution for mobile operating systems, as it ensures efficient memory management and reliable multitasking.

Question 9: Demand Paging in Online Video Editing Application

Case Analysis

An online video editing application loads only the required portions of large video files into memory. However, users experience delays while editing. The task is to analyze how **demand paging** affects performance and suggest methods to reduce page faults without increasing RAM capacity.

Analysis

- **Demand Paging** loads pages into memory only when they are required, reducing initial memory usage.
- Frequent **page faults** while accessing large video files increase disk access and slow down editing performance.

Recommended Solution

Use **Demand Paging** with an efficient **LRU or Clock page replacement algorithm**. Apply **page prefetching** and optimize page size to reduce page faults and improve access speed.

Justification

This approach reduces disk I/O, improves editing speed, decreases page faults, and enhances overall system performance without requiring additional RAM.

Conclusion

Using **Demand Paging** with efficient page replacement and prefetching provides faster video editing, better memory utilization, and improved application performance.

Question 10: Page Replacement Algorithms

Case Analysis

A data analytics server experiences frequent page faults while processing large datasets. The same pages are repeatedly loaded and replaced. The task is to compare **FIFO, LRU, Optimal, and Clock page replacement algorithms** and recommend the most suitable one.

Analysis

- **FIFO** replaces the oldest page but may remove frequently used pages.
- **LRU (Least Recently Used)** replaces the page that has not been used for the longest time, reducing page faults.
- **Optimal** replaces the page that will not be used for the longest time in the future, but it is not practical because future references are unknown.
- **Clock** is an efficient approximation of LRU with lower overhead.

Recommended Solution

Implement the **LRU page replacement algorithm** because it effectively reduces page faults for workloads with repeated memory access patterns. If implementation cost is a concern, **Clock** is a good alternative.

Justification

LRU improves memory utilization, reduces unnecessary page replacements, increases CPU efficiency, and provides better performance for large data processing.

Conclusion

LRU is the most suitable page replacement algorithm for this server because it minimizes page faults and improves overall system performance.

Question 11: Thrashing in University Examination Server

Case Analysis

A university examination server crashes during online exams due to excessive **page faults**. The task is to determine whether **thrashing** is the cause by analyzing CPU utilization, page fault frequency, and working set size, and then propose a memory allocation policy to eliminate it.

Analysis

- **Thrashing** occurs when the system spends more time swapping pages than executing processes.
- It results in **high page fault frequency**, **low CPU utilization**, and poor system performance.
- An insufficient **working set size** forces continuous page replacement, leading to server slowdown or crashes.

Recommended Solution

Implement the **Working Set Model** and **Page Fault Frequency (PFF)** control to allocate enough memory for active processes. Limit the number of concurrent processes to reduce excessive paging.

Justification

This approach minimizes page faults, improves CPU utilization, prevents thrashing, and ensures stable performance during online examinations.

Conclusion

Using the **Working Set Model** with **Page Fault Frequency control** is the best solution to eliminate thrashing and maintain reliable server performance during peak loads.

Question 12: Working Sets in Cloud Environment

Case Analysis

A cloud provider hosts hundreds of virtual machines (VMs) on a single physical server. During peak hours, several VMs become unresponsive. The task is to analyze the role of **working sets** in preventing thrashing and propose a dynamic memory allocation framework.

Analysis

- The **Working Set** is the set of memory pages currently required by a process.
- If a VM does not receive enough memory for its working set, **thrashing** occurs, reducing CPU utilization and slowing system performance.
- Dynamically adjusting memory based on workload improves efficiency.

Recommended Solution

Implement a **dynamic working set allocation** framework that continuously monitors VM memory usage and allocates or reclaims memory according to workload demands. Use **Page Fault Frequency (PFF)** to optimize memory distribution.

Justification

This approach prevents thrashing, improves CPU utilization, ensures smooth VM performance, and maximizes resource utilization during peak workloads.

Conclusion

Using a **dynamic working set model** with adaptive memory allocation is the most effective solution for maintaining high performance and preventing thrashing in cloud environments.

Question 13: Disk Devices and Virtual Memory Performance

Case Analysis

A multimedia company stores high-definition video files on **HDDs**, while frequently accessed metadata is stored on **SSDs**. The task is to analyze how disk devices affect **virtual memory performance, swapping speed, and page replacement efficiency**, and recommend a suitable storage hierarchy.

Analysis

- **HDDs** provide large storage capacity but have slower data access speeds.
- **SSDs** offer faster read/write operations, improving virtual memory access and reducing swapping time.
- Storing frequently used data on SSDs improves page replacement efficiency and overall system performance.

Recommended Solution

Use a **hybrid storage hierarchy**, where **SSDs** store virtual memory, swap space, and frequently accessed metadata, while **HDDs** store large video files and backup data.

Justification

This approach reduces access latency, speeds up swapping, improves page replacement performance, and enhances the efficiency of multimedia applications.

Conclusion

A **hybrid SSD-HDD storage hierarchy** provides the best balance of speed, storage capacity, and cost, resulting in improved virtual memory performance and faster application response times.

Question 14: Integrated Memory Management System

Case Analysis

A software company plans to redesign its operating system by integrating **dynamic linking, demand paging, segmentation, virtual memory, and SSD-based swapping**. The task is to evaluate this approach, identify possible bottlenecks, and suggest improvements.

Analysis

- **Dynamic Linking** reduces executable size and enables library sharing.
- **Demand Paging** loads pages only when required, improving memory utilization.
- **Segmentation** organizes memory into logical units for better protection.
- **Virtual Memory** allows execution of large applications beyond physical RAM.
- **SSD-based Swapping** speeds up page swapping and reduces disk access time.

Recommended Solution

Integrate all these memory management techniques with **LRU page replacement**, **Translation Lookaside Buffer (TLB)** for faster address translation, and **working set management** to prevent thrashing.

Justification

This integrated approach improves memory utilization, reduces loading time, minimizes disk I/O, enhances system performance, and supports efficient multitasking.

Conclusion

An integrated memory management system using **dynamic linking, demand paging, segmentation, virtual memory, and SSD-based swapping** provides high performance, efficient memory usage, and better scalability for modern operating systems.

Question 15: Dynamic Linking, Demand Paging, and Swapping

Case Analysis

An e-commerce platform experiences inconsistent response times during festival sales due to **excessive dynamic library loading, frequent page replacements, and swapping activity**. The task is to analyze their interaction and develop an optimization strategy that improves performance while minimizing memory consumption.

Analysis

- **Dynamic Linking** reduces executable size by loading shared libraries only when needed.
- **Demand Paging** loads required pages into memory on demand, improving memory utilization.
- **Swapping** moves processes between RAM and disk when memory is insufficient, increasing response time if performed frequently.

Recommended Solution

Use **Dynamic Linking with Lazy Loading**, implement **Demand Paging** with the **LRU page replacement algorithm**, and reduce unnecessary swapping by maintaining active processes in RAM using **virtual memory**.

Justification

This strategy reduces memory usage, minimizes page faults and swapping, improves application response time, and ensures smooth performance during high user traffic.

Conclusion

Combining **Dynamic Linking, Demand Paging, and efficient Swapping management** provides better memory utilization, faster response time, and improved performance for the e-commerce platform during peak loads.

Question 16: Memory Management for Scientific Simulation

Case Analysis

A research laboratory develops a scientific simulation that requires **large matrices exceeding available RAM**. The task is to compare **Virtual Memory, Segmentation, Paging, and Segmentation with Paging** and recommend the most efficient memory management model considering scalability, performance, and memory protection.

Analysis

- **Virtual Memory** allows large programs to execute even when RAM is limited.
- **Paging** eliminates external fragmentation and improves memory utilization.
- **Segmentation** organizes memory into logical sections and provides better protection.
- **Segmentation with Paging** combines the benefits of both techniques, offering efficient memory management and enhanced protection.

Recommended Solution

Implement **Segmentation with Paging** along with **Virtual Memory**. This combination supports large applications, improves memory utilization, provides better protection, and ensures efficient execution.

Justification

This approach offers high scalability, reduces fragmentation, improves memory security, and delivers better performance for large scientific computations.

Conclusion

Using **Segmentation with Paging** integrated with **Virtual Memory** is the most effective memory management model for handling large-scale scientific simulations efficiently.

Question 17: Memory Protection Between User and Kernel Processes

Case Analysis

A cybersecurity organization requires **strict memory protection** between user processes and kernel processes. The task is to analyze how **Segmentation, Paging, and Segmentation with Paging** provide memory isolation and design a secure memory architecture.

Analysis

- **Segmentation** separates memory into logical segments, providing protection for different processes.
- **Paging** divides memory into fixed-size pages, preventing unauthorized memory access.
- **Segmentation with Paging** combines both techniques to provide better memory isolation, protection, and efficient memory management.

Recommended Solution

Implement **Segmentation with Paging** and enforce separate memory spaces for **user** and **kernel** processes. Use memory protection bits and access permissions to restrict unauthorized access.

Justification

This approach improves memory security, prevents illegal memory access, enhances process isolation, and maintains efficient system performance.

Conclusion

Using **Segmentation with Paging** is the most secure and efficient memory management approach, ensuring strong protection between user and kernel processes while maintaining high execution efficiency.

Question 18: Memory Management Framework for Financial Servers

Case Analysis

A fintech company is designing a next-generation operating system for financial servers. The system must support **secure memory allocation, efficient page replacement, fast loading of shared libraries, and minimal disk I/O**. The task is to develop an integrated memory management framework.

Analysis

- **Dynamic Storage Management** allocates memory efficiently based on process needs.
- **Paging** and **Virtual Memory** improve memory utilization and support multitasking.
- **Working Set Model** prevents thrashing by maintaining active pages in memory.
- **SSD Storage** speeds up swapping and reduces disk I/O.
- **Dynamic Linking** enables fast loading of shared libraries.

Recommended Solution

Implement a framework combining **Dynamic Storage Management, Paging, Virtual Memory, Working Set Model, Dynamic Linking, and SSD-based Swap Space**. Use the **LRU page replacement algorithm** to reduce page faults.

Justification

This integrated framework improves memory security, minimizes disk access, reduces page faults, enhances application performance, and ensures reliable operation of financial servers.

Conclusion

A memory management framework integrating **Dynamic Storage Management, Paging, Virtual Memory, Working Set Model, Dynamic Linking, and SSD storage** provides secure, efficient, and high-performance memory management for financial systems.

Question 19: Memory Management for Autonomous Vehicle

Case Analysis

An autonomous vehicle runs **perception, navigation, and object detection** modules simultaneously on limited embedded memory. The task is to analyze how **Demand Paging, Page Replacement, Virtual Memory, and Working Set Model** affect real-time performance and recommend a suitable memory management strategy.

Analysis

- **Demand Paging** loads only required pages, reducing memory usage.
- **Virtual Memory** allows applications to run efficiently with limited RAM.
- **Page Replacement (LRU/Clock)** minimizes unnecessary page faults.
- **Working Set Model** keeps frequently used pages in memory, preventing thrashing and ensuring smooth execution.

Recommended Solution

Implement **Demand Paging** with the **Working Set Model**, use the **Clock page replacement algorithm**, and allocate sufficient memory to critical modules such as perception and navigation to guarantee real-time performance.

Justification

This strategy reduces page faults, minimizes latency, prevents thrashing, and ensures reliable execution of safety-critical applications.

Conclusion

Using **Demand Paging, Virtual Memory, the Working Set Model, and the Clock page replacement algorithm** provides efficient memory management and deterministic real-time performance for autonomous vehicles.

Question 20: Memory Management Architecture for AI-Powered Supercomputers

Case Analysis

A technology company plans to build an operating system for **AI-powered supercomputers** that supports millions of concurrent processes, shared libraries, heterogeneous memory devices, SSDs, and distributed storage. The task is to design a complete memory management architecture and evaluate its performance, scalability, reliability, and cost.

Analysis

- **Dynamic Linking** enables efficient sharing of libraries.
- **Paging** and **Virtual Memory** improve memory utilization and support large-scale multitasking.
- **Segmentation with Paging** provides memory protection and logical organization.
- **Demand Paging** and the **Working Set Model** reduce page faults and prevent thrashing.
- **SSD-based Swapping** and advanced disk management improve data access speed.

Recommended Solution

Design an integrated memory management architecture combining **Dynamic Linking, Paging, Segmentation with Paging, Virtual Memory, Demand Paging, Working Set Model, LRU Page Replacement, and SSD-based Swapping** to achieve high performance and scalability.

Justification

This architecture improves memory utilization, reduces disk I/O, supports millions of concurrent processes, enhances system reliability, and provides a balance between performance and operational cost.

Conclusion

An integrated memory management architecture using **Dynamic Linking, Paging, Segmentation with Paging, Virtual Memory, Demand Paging, Working Set Management, and SSD storage** is the most suitable solution for future AI-powered supercomputers because it ensures **high performance, scalability, reliability, and efficient resource utilization**.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts

Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand